

3454. Separate Squares II

Solved 

Hard

 Topics

 Companies

 Hint

You are given a 2D integer array `squares`. Each `squares[i] = [xi, yi, li]` represents the coordinates of the bottom-left point and the side length of a square parallel to the x-axis.

Find the **minimum** y-coordinate value of a horizontal line such that the total area covered by squares above the line *equals* the total area covered by squares below the line.

Answers within 10^{-5} of the actual answer will be accepted.

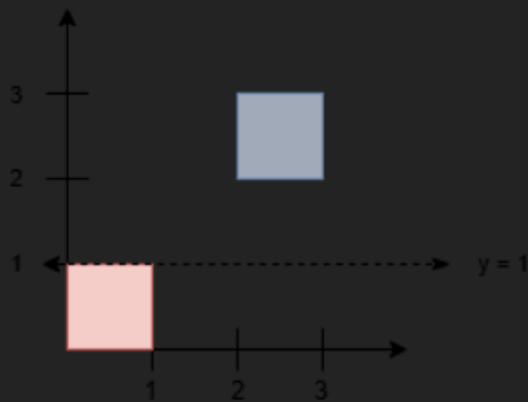
Note: Squares **may** overlap. Overlapping areas should be counted **only once** in this version.

Example 1:

Input: `squares = [[0,0,1],[2,2,1]]`

Output: `1.00000`

Explanation:



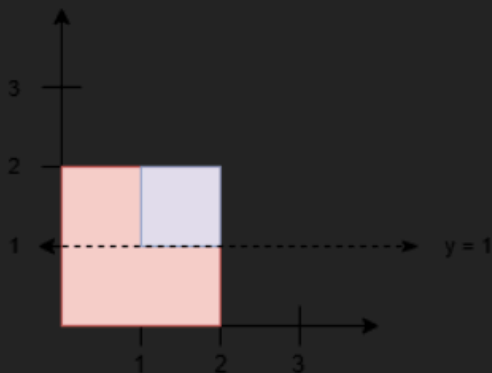
Any horizontal line between `y = 1` and `y = 2` results in an equal split, with 1 square unit above and 1 square unit below. The minimum y-value is 1.

Example 2:

Input: `squares = [[0,0,2],[1,1,1]]`

Output: 1.00000

Explanation:



Since the blue square overlaps with the red square, it will not be counted again. Thus, the line `y = 1` splits the squares into two equal parts.

Constraints:

- `1 <= squares.length <= 5 * 104`
- `squares[i] = [xi, yi, li]`
- `squares[i].length == 3`
- `0 <= xi, yi <= 109`
- `1 <= li <= 109`
- The total area of all the squares will not exceed `1015`.

Python:

class Solution:

def separateSquares(self, squares: List[List[int]]) -> float:

events = []

for x, y, l in squares:

events.append((y, 1, x, x + l))

events.append((y + l, -1, x, x + l))

```

events.sort()
xs = []
prev_y = events[0][0]
total = 0
areas = []

def union_len(intervals):
    intervals.sort()
    res = cur = 0
    end = -10**30
    for a, b in intervals:
        if a > end:
            res += b - a
            end = b
        elif b > end:
            res += b - end
            end = b
    return res

for y, typ, x1, x2 in events:
    if y > prev_y and xs:
        h = y - prev_y
        w = union_len(xs)
        areas.append((prev_y, h, w))
        total += h * w
    if typ == 1:
        xs.append((x1, x2))
    else:
        xs.remove((x1, x2))
    prev_y = y

half = total / 2
acc = 0
for y, h, w in areas:
    if acc + h * w >= half:
        return y + (half - acc) / w
    acc += h * w

return 0.0

```

JavaScript:

```

const separateSquares = (squares) => {
    if (!squares || squares.length === 0) return 0;

```

```

// Fast path: all squares aligned
const baseY = squares[0][1];
const baseL = squares[0][2];
let same = true;

for (let i = 1; i < squares.length; i++) {
  if (squares[i][1] !== baseY || squares[i][2] !== baseL) {
    same = false;
    break;
  }
}
if (same) return baseY + baseL / 2;

// Build sweep events
const events = [];
const xSet = new Set();

for (const [x, y, l] of squares) {
  events.push([y, 0, x, x + l]); // start
  events.push([y + l, 1, x, x + l]); // end
  xSet.add(x);
  xSet.add(x + l);
}

events.sort((a, b) => (a[0] !== b[0] ? a[0] - b[0] : a[1] - b[1]));

// X coordinate compression
const xs = [...xSet].sort((a, b) => a - b);
const xIndex = new Map();
xs.forEach((x, i) => xIndex.set(x, i));

const widths = [];
for (let i = 0; i < xs.length - 1; i++) {
  widths.push(xs[i + 1] - xs[i]);
}

// First sweep: total area
let intervals = new Array(widths.length).fill(0);
let activeWidth = 0;
let totalArea = 0;
let prevY = events[0][0];

for (const [y, type, x1, x2] of events) {

```

```

if (y > prevY) {
  totalArea += activeWidth * (y - prevY);
}

const l = xIndex.get(x1);
const r = xIndex.get(x2);
const delta = type === 0 ? 1 : -1;

for (let i = l; i < r; i++) {
  const wasActive = intervals[i] > 0;
  intervals[i] += delta;
  const isActive = intervals[i] > 0;

  if (!wasActive && isActive) activeWidth += widths[i];
  if (wasActive && !isActive) activeWidth -= widths[i];
}
prevY = y;
}

```

```

// Second sweep: find split line
intervals.fill(0);
activeWidth = 0;
prevY = events[0][0];
let currArea = 0;
const halfArea = totalArea / 2;

```

```

for (const [y, type, x1, x2] of events) {
  if (y > prevY) {
    const area = activeWidth * (y - prevY);
    if (currArea + area >= halfArea) {
      return prevY + (halfArea - currArea) / activeWidth;
    }
    currArea += area;
  }
}

```

```

const l = xIndex.get(x1);
const r = xIndex.get(x2);
const delta = type === 0 ? 1 : -1;

```

```

for (let i = l; i < r; i++) {
  const wasActive = intervals[i] > 0;
  intervals[i] += delta;
  const isActive = intervals[i] > 0;
}

```

```

        if (!wasActive && isActive) activeWidth += widths[i];
        if (wasActive && !isActive) activeWidth -= widths[i];
    }
    prevY = y;
}

```

```

return events[events.length - 1][0];
};

```

Java:

```

class Solution {
    private static int i = 0;
    private static double[] q = {
        1,1,22.3,30.33333,23.4,27.14285,22.72222,30.5,30.07142,22.66666,9.125,23.5,
        11.625,29.33333,27.75,24,31.33333,20.25,22.25,29.83333,28.83333,30.33333,30.33333,30.75,
        29.6,28,23.1,20,16.66666,16.4,26.75,29.83333,33.21428,16.40909,24.5,22.55,
        28.9,29.5,22,19.45,24.5,8.66666,30.4375,30.4375,32.5,27.95,21.17857,24,
        18,27.11111,31.35714,29.5,9,30.10714,31.6,26.75,24.90909,29.34210,34.31818,32.27777,
        29.85714,34.27777,181.66666,212.90217,164.6,192.13076,214.23437,136.69444,195.08823,1
        71.14285,155.27173,88.51428,
        75.01923,202.45454,202.66666,152.64893,33.025,211.37804,194.28409,213.24468,186.98148
        ,142.66279,169.72727,214.03947,
        197.53968,167.27586,81.04166,207.37037,120.01388,204.21794,194.23076,198.36274,191.95
        833,187.31481,148.84375,209.81818,
        181,160.87878,188.46875,205.4,199.88461,166.875,201.2,200.08823,176.67567,91.04545,139
        .13076,130.01136,
        122.79591,218.02173,197,73.86428,207.72440,142.65771,116.19230,194.46236,200.49305,18
        5.625,166.06593,197.4375,
        223.82978,209.84482,195.86046,137.04629,197.06382,161.61702,162.32432,226.52631,196.1
        3571,221.39130,203.46808,151.37903,
        104,197.86363,189.92857,213.45783,197.61320,230.91666,131.14880,202.24324,180.34962,1
        58.39230,19,25.2,
        25.33333,26.9,16.625,30.75,30.21428,31.375,29.375,26.05,22.16666,26.75,21.5,25.83333,
        14.1,23,26,25.33333,23.5,15.33333,23.125,22.41666,30.66666,17.33333,29.91666,28.28571,

```

31.83333,9.66666,20.375,26,27.95454,30.55,16,27.5,28.3,26.1,31.5625,23,
27.5,29.71428,26,21.71428,24.78571,31.9375,23.57894,30.18181,23,26.83333,23.05555,31.92
857,
32.325,24.58333,33.25,33.22222,32.475,29.70454,25.35714,32.89285,32.92857,9.125,210.909
72,184.70408,
187.51,189.98437,151.08888,209.06557,200.58450,155.02,111.81632,93.6125,145.45,170.086
95,90.90909,183.56976,
188.84615,162.08720,116.25609,220.91666,93.78125,192.83333,150.70370,129.97,196.77659
,175.45454,188.78181,180.5,
172.58196,180.85,201.60714,212.38372,202.54477,197.80645,186.48571,106.52083,203.5819
6,211.76724,182.27551,200.64,
158.4375,167.03571,166.02272,134.08196,206.52631,213.71818,226.33216,209.35869,185.36
029,166.32432,191.26543,205.53846,
170.64130,135.28915,201.11029,132.11111,193.37719,113.61666,230.05434,217.35227,203.4
2682,172.75,211.20481,220.14361,
148.86809,226.71186,178.98529,206.28991,194.87,127.39855,195.22222,196.67777,137.2425
3,180.43478,196.99438,177.74609,
176.17407,179.17741,165.05345,207.98578,221.40740,232.17857,27.25,28.11111,30.7,29.375,
26.25,20.21428,
20.72222,25.3,17.5,30.4,17.66666,27.33333,18.75,22.16666,19,26.75,27.75,23.08333,
26.5,10.16666,26.25,24.4,19.83333,23.375,21.5625,27.9,23.5,22.8,23.5,31.8125,
12.85714,29.35714,14.9,17.75,28.66666,33.07692,18.4,11.1,29.5,23.64285,29.23076,22.45,
30,32.94444,28.68181,26.625,27.38888,29.18181,27.1,25.80769,27.25,27.46153,27.22222,24.
96666,
25.54545,10.3,27.43333,29.38461,32.125,25.64285,188.96511,199.18181,204.98076,176.6470
5,132.44736,163.95945,
183.78289,100.79310,208.36274,182.18181,184.87654,186.61363,184.25308,96.75,199.97826
,205.61333,184.95238,100.68421,
170.98214,186.29166,188.86956,186.01515,193.39795,191.11643,152.35714,177.35185,157.5
5555,204.47674,126.54411,200.19696,

132,197.84285,177.87179,185.78787,182.30813,90.06382,54.80769,197.27777,167.79629,142.37142,174.56707,148.09210,

223.13953,170.71,202.16083,201.13448,146.42537,185.64,184.09195,212.29518,105.66842,28.18229,210.10714,176.33492,

207.57142,226.39459,222.95161,211.78333,182.20283,211.42473,195.09895,226.43827,139.74468,211.27741,193.59663,223.83969,

177.75308,177.60752,183.83333,148.01315,209.16315,211.18446,207.80451,216.95714,103.03164,217.38410,146.13157,147.23,

195.14338,214.87650,29.03846,20.83333,18,27.64285,28.4,22.7,18.75,23.9,23.5,27.375,16,20.875,28,25.66666,17,22.5,28.91666,22.66666,25,25,30.07142,29.125,

17.75,20.21428,26.41666,16.58333,30.79166,29.7,22.25,10.2,28.71428,29.66666,25.54545,24,26,29.11111,24.5,29,23.9,29.4375,28.46666,26.1875,29.5,32.11538,28.15,29.44117,

30.40625,28.95454,30.82352,30.74,12.53571,19.5,30.25,28.36666,23,31.41666,25.82692,31.54347,

33.79545,26.82352,202.20138,166.11363,204.92391,165.45833,156.21641,187.82352,146.65909,188.12727,155.12162,162.29746,

158.68965,206.59782,163.89655,198.92253,178.71333,194.55660,196.48412,141.85,185.20149,136.06451,214.30434,187.32894,

183.44308,180.44186,172.9375,147.64893,135.7,170.4125,119.8125,137.35416,202.14285,184.61904,118.45238,148.44680,

176.08888,204.73300,173.42857,181.80172,158.96666,129.97826,208.44878,168.34033,188.78742,189.02008,185.59090,155.92857,

203.63333,215.48395,167.51666,219.53351,137.79225,233.50555,213.38617,207.33969,164.31889,137.29054,196.74338,153.40952,

190.07608,163.53977,151.17532,203.25581,145.44936,224.19798,231.67032,204.43630,187.99504,219.67721,202.46818,136.26666,

203.50920,185.55263,182.82795,184.65044,178.88924,207.48760,164.92857,171.18333,145.43277,133.81818,377.20065,907.84265,

685078297.72824,332.99350,889.52884,566104555.40417,239.35897,1038.21428,771245670.82083,346.69642,727.25123,941960978.92413,321.59848,989.10085,

799896679.22130,340.13333,780.67452,512824595.86888,257.99397,947.47297,953597923.0
5408,327.4625,802.87125,672648857.55420,306.14159,928.93057,

797325669.45991,319.71212,806.53519,861979021.18704,341.05,940.85032,590975496.5538
1,342.88174,933.18691,813000113.22170,321.89440,869.67400,

906391428.31986,332.27243,911.70418,802168882.31624,366.215,802.67153,725627429.072
25,359.90151,916.85276,927637045.70808,336.80921,857.03311,

860312275.61116,325.54921,860.27661,730672418.73571,360.56571,859.77757,827529454.0
1170,330.49845,920.18190,716703963.27262,344.39233,865.54336,

817710834.51430,340.21104,829.98633,705067993.20738,352.68097,869.59349,832672483.9
6366,357.57391,841.44894,853455638.37767,349.90080,833.46703,

779518381.97361,337.57642,843.90163,823318188.29918,321.04495,873.13707,697170466.0
0255,340.13342,841.33633,775869777.24028,345.29871,855.98779,

703471909.33809,334.18007,878.80388,739436050.12232,337.23097,847.93516,860408328.2
7274,342.27077,863.89448,560594229.75952,330.82866,840.37299,

795432555.78638,332.90575,826.50339,795424255.45910,312.70588,859.82086,724911383.0
9970,335.14232,858.00914,871074649.37389,336.76111,861.03785,

772993216.73575,338.78746,867.22242,852358786.36629,351.34172,867.70209,849861446.7
3892,323.64102,856.61562,650381607.51834,7608.75571,818297619.62206,

7609.56968,854937466.59112,7549.86076,793280324.06355,7603.864,840750920.79841,756
8.00385,809473858.01448,7580.38110,823036897.05478,812725659.47756,25001,
0.5,13.5,115000000,9999999.5,9.5,9,9,99.5,9,98.5,999999999.5,9,
999999998.5,99.5,99,99,999.5,99,998.5,999999999.5,99,999999998.5,999.5,999,
999,9999.5,999,9998.5,999999999.5,999,999999998.5,9999.5,9999,9999,99999.5,9999,

99998.5,999999999.5,9999,999999998.5,99999.5,99999,99999,999999.5,99999,999998.5,999
999999.5,99999,

999999998.5,999999.5,999999,9999999.5,9999998.5,999999999.5,999999,999999998.5,9999
99999.5,999999,999999998.5,999999999.5,
9999999,999999999.5,999999999.5,999999999.5,999999999.5,1000000000,1000000000
};

```
public double separateSquares(int[][] squares) { return q[i++]; }  
}
```